Lecture 16: Transformer Architecture

Week 5, Lecture 2 - Attention Is All You Need

PSYC 51.07: Models of Language and Communication

Winter 2026

Today's Agenda 📋

1. 🤖 **The Transformer Revolution**: Why it changed everything
2. 🏗️ **Architecture Overview**: Encoder, Decoder, and components
3. 🎯 **Self-Attention**: The core mechanism (Q, K, V)
4. 📝 **Self-Attention Example**: Understanding pronoun resolution
5. 🎭 **Multi-Head Attention**: Learning diverse relationships
6. 🔍 **Three Types of Attention**: Self, Masked, Cross

Goal: Understand the Transformer architecture and self-attention

The Transformer Revolution 🤖

"Attention Is All You Need"

Before Transformers (2017):

- RNNs/LSTMs with attention
- Sequential processing
- Hard to parallelize
- Limited context window
- Slow training

After Transformers:

- Parallel processing
- Scales to GPUs/TPUs

Why Get Rid of RNNs? 🚫

Limitations of Recurrent Architectures:

1. Sequential Bottleneck

- Must process token t before t+1
- Cannot parallelize across sequence

2. Long-Range Dependencies

- Info flows through many steps
- Gradient vanishing problems

3. Memory Constraints

- Hidden state must remember all

Concrete Example:

Transformer Architecture Overview 🏗️

```
1**Encoder -> Feed Forward -> Multi-Head Attention -> Feed Forward -> Multi-Head
```

\end{center**

Key Components:

- **Multi-Head Self-Attention**: Relate all positions to each other
- **Feed-Forward Networks**: Transform representations
- **Residual Connections & Layer Norm**: Training stability (not shown)
- **Positional Encoding**: Inject position information

Self-Attention: The Core Mechanism 🎯

Key idea: Each word attends to all other words in the sequence

Three learned projections:

- **Query (Q)**: What am I looking for?
- **Key (K)**: What do I contain?
- **Value (V)**: What information do I have?

Computation:

```
1Q = X @ W_Q    # Transform input to queries
2K = X @ W_K    # Transform input to keys
3V = X @ W_V    # Transform input to values
4
5Attention(Q, K, V) = softmax(Q @ K.T / sqrt(d)) @ V
```

Understanding Query, Key, Value 🔑

Analogy: Database lookup or information retrieval

Information Retrieval:

- **Query**: Your search query
- **Key**: Document titles/keywords
- **Value**: Document contents

Process:

1. Compare query to all keys
2. Get similarity scores
3. Weight values by scores
4. Return weighted combination

Scaled Dot-Product Attention 📐

Step-by-step computation with concrete example:

Input: 3 tokens, embedding dim = 4

```
# Input embeddings (3 tokens x 4 dims)
X = [[0.1, 0.2, 0.3, 0.4],   # "The"
     [0.5, 0.6, 0.7, 0.8],   # "cat"
     [0.2, 0.3, 0.4, 0.5]]   # "sat"

# Step 1: Compute Q, K, V (using learned weights W_q, W_k, W_v)
Q = X @ W_q  # [3 x 4]
K = X @ W_k  # [3 x 4]
V = X @ W_v  # [3 x 4]

# Step 2: Compute attention scores
scores = Q @ K.T  # [3 x 3] - each token vs each token

# Step 3: Scale by sqrt(d_k) to prevent large values
```

Scaled Dot-Product Attention 📐

```
21output = weights @ V  # [3 x 4] - new contextual embeddings
```

...continued

Self-Attention Example: Pronoun Resolution 📝

Sentence: "The animal didn't cross the street because it was too tired"

Question: What does "it" refer to?

```
1Attention weights when processing "it":
2
3          The  animal  didn't  cross  the  street  because  it   was  too  tir
4"it" →    0.02  [0.45]   0.03   0.05  0.02  0.08    0.05   0.15  0.05 0.02  0.0
5                  ↑
6          High attention to "animal" - model learns coreference!
```

Self-attention allows the model to:

- Resolve pronouns ("it" → "animal", not "street")
- Understand long-range dependencies (8 tokens apart!)
- Capture syntactic and semantic relationships

Visualizing the Attention Matrix 🔍

For sentence: "The cat sat on the mat"

To →	The	cat	sat	on	the	mat
From ↓						
cat	0.1	0.5	0.2	0.1	0.05	0.05
sat	0.05	0.3	0.4	0.15	0.05	0.05
on	0.05	0.1	0.2	0.3	0.1	0.25
the	0.05	0.05	0.05	0.1	0.3	0.45
mat	0.05	0.05	0.1	0.2	0.2	0.4

Observations:

Multi-Head Attention 🎭

Why use multiple attention heads?

Intuition:

- Different heads learn different relationships
- Head 1: Syntactic relationships
- Head 2: Semantic relationships
- Head 3: Positional patterns

Formula:

```
1# Each head has its own W_Q, W_K, W_V
2head_1 = Attention(Q @ W1_Q, K @ W1_K, V @ W1_V)
3head_2 = Attention(Q @ W2_Q, K @ W2_K, V @ W2_V)
4# ... more heads ...
```

Why Multiple Heads? 🤔

Example: Different heads learn different patterns

Sentence: "The cat sat on the mat"

Head 1: Syntactic Dependencies

- "cat" → "The" (noun-determiner)
- "sat" → "cat" (verb-subject)
- "mat" → "the" (noun-determiner)
- Learns grammar structure

Head 2: Semantic Relations

- "sat" → "mat" (action-location)
- "cat" → "mat" (agent-location)

Three Types of Attention 🔍

1. **Self-Attention (Encoder)**
    - Each position attends to all positions in same sequence

- Bidirectional: can see past and future
- Used in: BERT, encoder-only models

2. **Masked Self-Attention (Decoder)**
    - Each position attends only to previous positions

- Prevents "looking into the future"
- Used in: GPT, decoder-only models

3. **Cross-Attention (Encoder-Decoder)**
    - Decoder attends to encoder outputs

- Queries from decoder, Keys/Values from encoder

Masked Self-Attention 🎭

Preventing the model from "cheating" during generation

Problem: During training, we have the full target sequence. Without masking, the model could "peek" at future tokens!

Solution: Mask out future positions by setting attention scores to -infinity before softmax.

```
1# Example: Generating "The cat sat"
2# When predicting "sat", model should only see "The cat"
3
4scores = [[0.5, 0.3, 0.2],    # "The" can see: The
5          [0.4, 0.5, 0.1],    # "cat" can see: The, cat
6          [0.2, 0.4, 0.4]]    # "sat" can see: The, cat, sat
7
8# Apply causal mask (upper triangle = -infinity)
9mask = [[ 0,  -inf, -inf],
10        [ 0,   0,   -inf],
```

Cross-Attention 🔗

Connecting encoder and decoder in seq2seq models

```
1Encoder Outputs ->  (Keys & Values) -> Decoder State ->  (Queries) -> Cross-Att
```

Key Properties:

- **Q** comes from decoder (what decoder needs)
- **K, V** come from encoder (what input provides)
- Allows decoder to "look at" relevant parts of input
- Similar to the original attention mechanism from Lecture 12!

Used in: Machine translation, summarization, any encoder-decoder task

Implementing Self-Attention in PyTorch 💻

Scaled dot-product attention

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class SelfAttention(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.embed_dim = embed_dim
         self.W_q = nn.Linear(embed_dim, embed_dim)
         self.W_k = nn.Linear(embed_dim, embed_dim)
         self.W_v = nn.Linear(embed_dim, embed_dim)

    def forward(self, x, mask=None):
        Q = self.W_q(x)  # Queries: what am I looking for?
        K = self.W_k(x)  # Keys: what do I contain?
```

Implementing Self-Attention in PyTorch 💻

```
        scores = scores / math.sqrt(self.embed_dim)  # Scale!

        if mask is not None:  # For causal/decoder attention
            scores = scores.masked_fill(mask == 0, -1e9)

        attn_weights = F.softmax(scores, dim=-1)  # Normalize
        output = torch.matmul(attn_weights, V)    # Weighted sum

        return output, attn_weights

# Usage example:
attn = SelfAttention(embed_dim=64)
x = torch.randn(1, 5, 64)  # 5 tokens, 64-dim embeddings
out, weights = attn(x)
# out: [1, 5, 64] - contextualized embeddings
# weights: [1, 5, 5] - attention matrix
```

continued

Computational Complexity ⚙️

Understanding the cost of self-attention

Component	Time Complexity	Memory
Self-Attention	O(n^2 * d)	O(n^2)
Feed-Forward	O(n * d^2)	O(d)

where n = sequence length, d = embedding dimension

Concrete Example: Memory Usage

Sequence length n = 1000 tokens, d = 768 (BERT-base)

Attention matrix size: n x n = 1000 x 1000 = **1 million entries**

At fp32 (4 bytes): **4 MB** per layer, per head

BERT-base: 12 layers x 12 heads = 144 attention matrices

Discussion Questions 💭

1. **Self-Attention vs RNN Attention:**
    - What's the key difference?

- Why is self-attention more powerful?
- When might RNNs still be useful?

2. **Query, Key, Value Framework:**
    - Why three separate projections instead of one?

- What if we used $Q = K = V = X$?
- How does this relate to information retrieval?

3. **Multi-Head Attention:**
    - Why not just use one big attention head?

- How many heads is optimal?

Looking Ahead 🔮

What's Next?

Today we learned:

- Why transformers replaced RNNs
- Self-attention mechanism (Q, K, V)
- Multi-head attention
- Three types of attention (self, masked, cross)

Next lecture (Lecture 14 - Training Transformers):

- : How to inject position information
- : The other key component
- : Training stability

Summary 🎯

Key Takeaways:

1. **Transformer Revolution**
    - Pure attention, no recurrence
    - Parallel processing, faster training
2. **Self-Attention Mechanism**
    - Query, Key, Value framework
    - Each token attends to all others
    - Scaled dot-product: $(QK^T/)V$
3. **Multi-Head Attention**
    - Multiple heads learn diverse relationships

References 📚

Essential Papers:

- **Vaswani et al. (2017)** - "Attention Is All You Need"
- The original Transformer paper
- Introduced self-attention, multi-head attention
- Foundation of modern NLP

  \item **Bahdanau et al. (2015)** - "Neural Machine Translation by Jointly Learning to Align and Translate"

    - Original attention mechanism (for comparison)

Tutorials and Resources:

- **The Illustrated Transformer** by Jay Alammar

Questions? 🙋

Discussion Time

Topics for discussion:

- Self-attention mechanism
- Query, Key, Value intuition
- Multi-head attention
- Masked vs. unmasked attention
- Implementation questions

Thank you! 🙏

Next: Training Transformers!